

CSE332: Computer Architecture and Organization

Assignment#5

Submitted by,

Name: Muhtadina Serniabat Tasin

Student ID: 2232160642

Submitted to,

Faculty Initial: MAQm

Section: 5

Submission Date:

Spring 2025
North South University

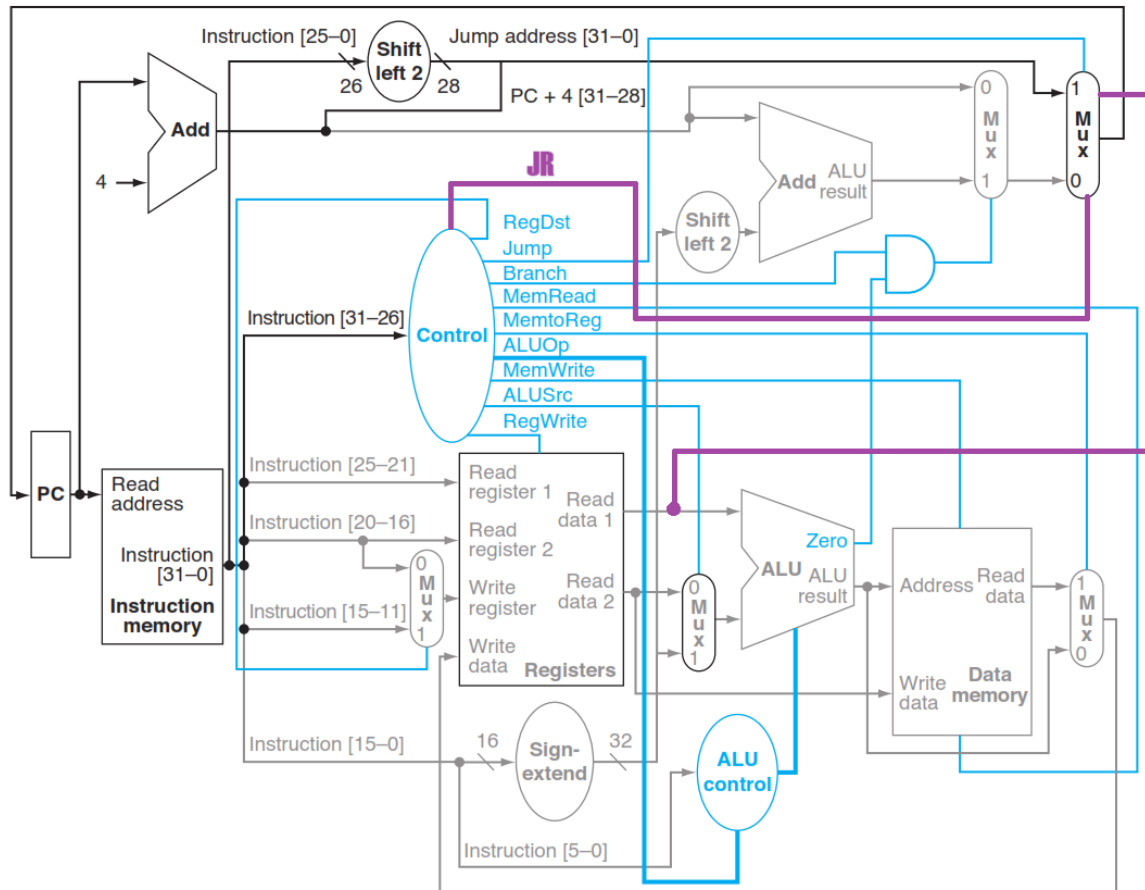
MIPS Architecture



The diagram illustrates the MIPS processor architecture with the following components and connections:

- PC (Program Counter):** Receives the next instruction address from the Mux 2 output. It provides a 4-bit constant to the Add block and the instruction address to the Instruction memory.
- Instruction Memory:** Receives the instruction address from the PC. It outputs instruction fields: [31-26] to the Control block, [25-21] to Read register 1, [20-16] to Read register 2, [15-11] to the Write register, and [15-0] to the Sign-extend block.
- Control:** Receives instruction fields [31-26] and outputs control signals: RegDst, Jump, Branch, MemRead, MemtoReg, ALUOp, MemWrite, ALUSrc, and RegWrite.
- Registers:** Contains Read register 1, Read register 2, and a Write register. It receives read data from the Data memory and write data from the ALU result. It outputs the 26-bit instruction field to the Shift left 2 block.
- Shift left 2:** Takes the 26-bit instruction field and outputs the 28-bit Jump address [31-0].
- Add:** Takes a 4-bit constant and the PC value to calculate PC + 4 [31-28]. It also receives the 26-bit instruction field via the JAL path.
- ALU:** Receives the ALUOp control signal and two 32-bit operands (one from the registers, one from the ALUSrc block). It outputs the ALU result to the Mux 1 and the Zero flag to the ALU control.
- ALUSrc:** Takes the ALUSrc control signal and the 16-bit sign-extended value to produce a 32-bit operand for the ALU.
- Mux 1:** Selects between the register value and the ALU result based on the MemtoReg control signal. Its output goes to the Write register and the Data memory.
- Mux 2:** Selects between the PC + 4 value and the ALU result based on the Jump control signal. Its output is the next PC value.
- Sign-extend:** Takes the 16-bit instruction field [15-0] and extends it to 32 bits.
- ALU control:** Takes the ALUOp control signal and the 5-bit instruction field [5-0] to generate the ALU control signal.

2. JR instruction:



3. SLL/SRL instructions:

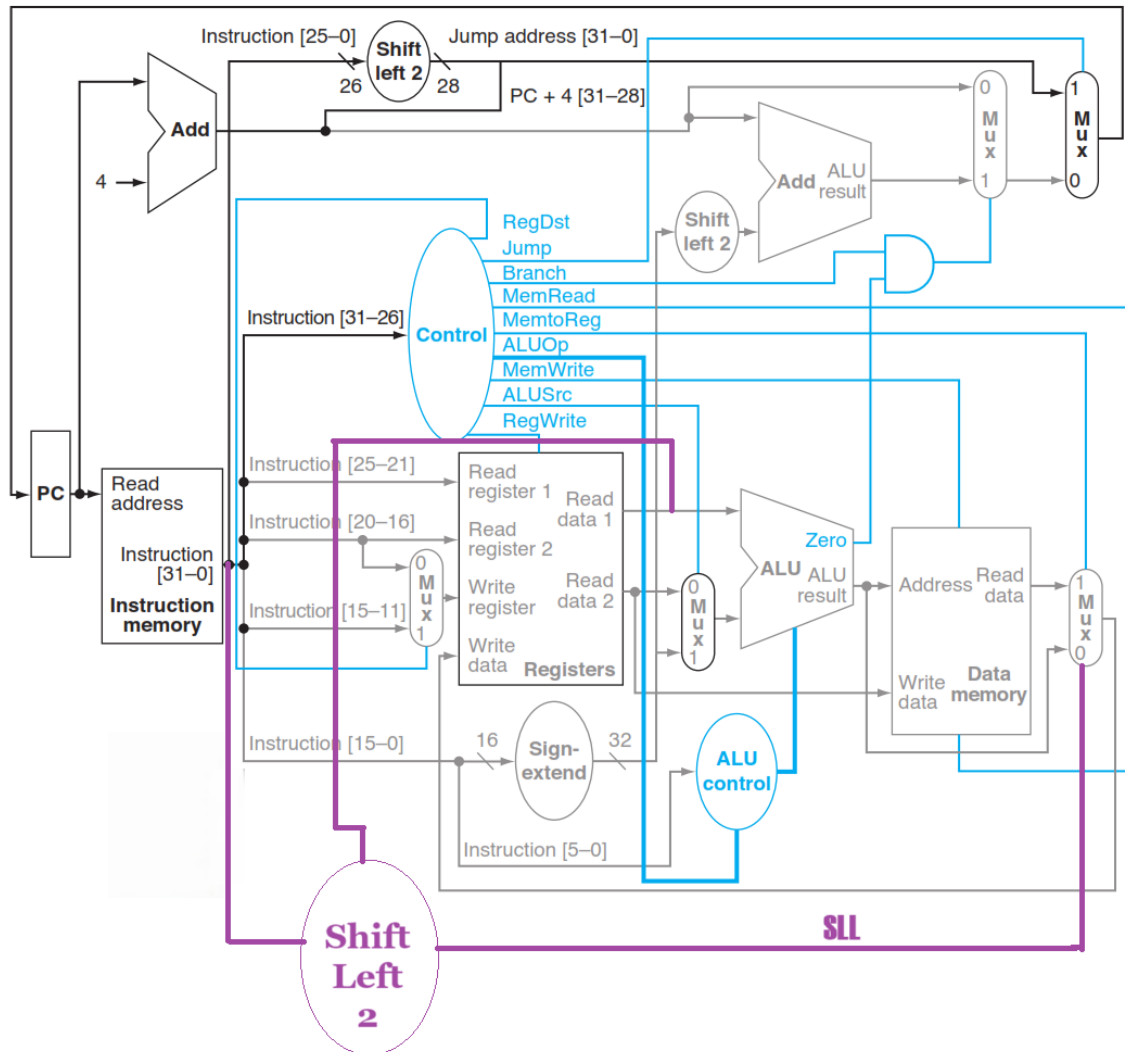


Fig – SLL instruction

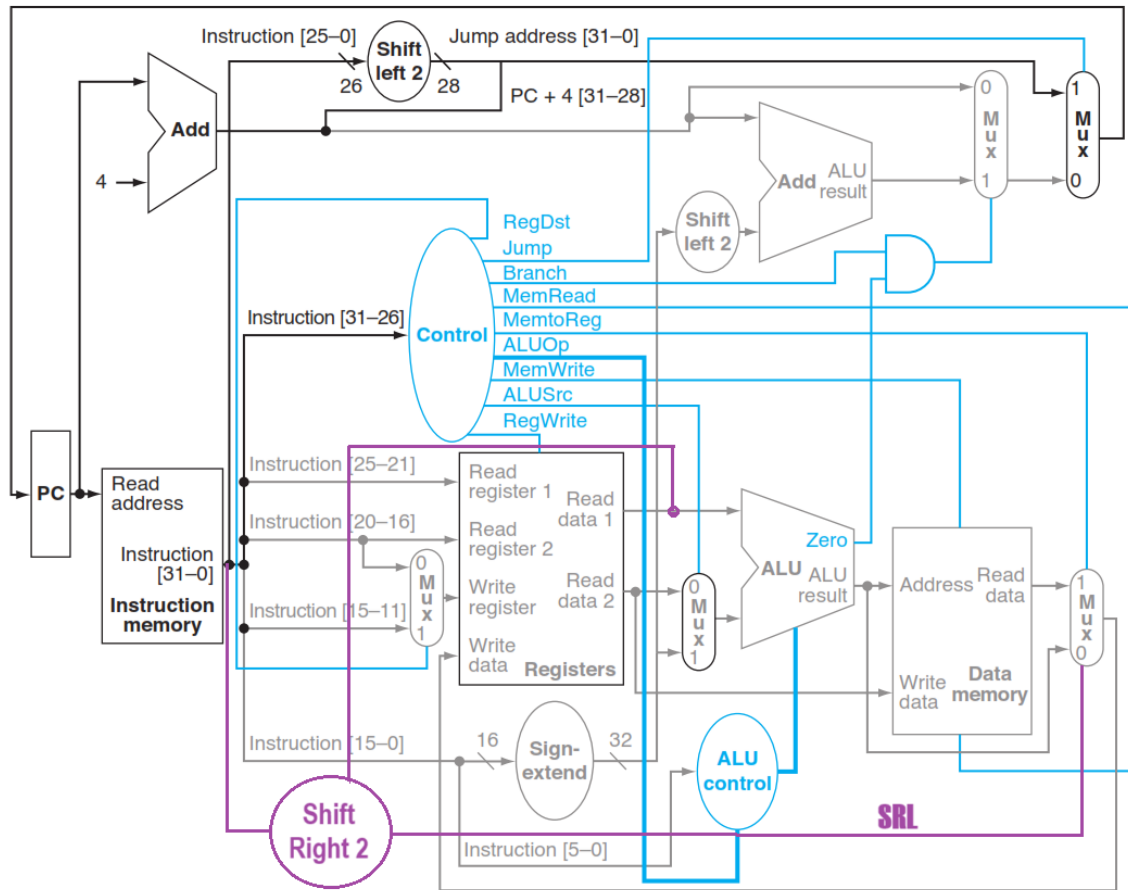


Fig – SRL Instruction

Control Signal Table

Instruction	RegDst	ALUSrc	MemToReg	RegWrite	MemRead	MemWrite	Branch	Jump	ALU OP[1]	ALU OP[0]	Special Notes
JAL	1	x	0	1	0	0	0	1	x	x	Write PC+4 to \$ra
JR	x	x	x	0	0	0	0	1	x	x	PC ← \$ra
SLL	1	0	0	1	0	0	0	0	1	0	Shift Left
SRL	1	0	0	1	0	0	0	0	1	0	Shift Right

Modified control.v

```
// modified file: control.v
`timescale 1ns/1ns
module Control_unit(input [5:0] Opcode,
    input [5:0] Func,
    input Zero,
    output reg MemtoReg,
    output reg MemWrite,
    output reg ALUSrc,
    output reg RegDst,
    output reg RegWrite,
    output reg Jump,
    output PCSrc,
    output reg [3:0] ALUControl,
    output reg JAL,    // New signal for JAL
    output reg JR      // New signal for JR
);
reg [7:0] temp;
reg Branch,BNE;

always @(*) begin
    // Default values
    JAL = 1'b0;
    JR = 1'b0;

    case (Opcode)
        6'b000000: begin // R-type
            temp <= 8'b11000000;
            case (Func)
                6'b100000: ALUControl <= 4'b0000; // ADD
                6'b100001: ALUControl <= 4'b0000; // ADDU
                6'b100010: ALUControl <= 4'b0001; // SUB
                6'b100011: ALUControl <= 4'b0001; // SUBU
                6'b100100: ALUControl <= 4'b0010; // AND
                6'b100101: ALUControl <= 4'b0011; // OR
                6'b100110: ALUControl <= 4'b0100; // XOR
                6'b100111: ALUControl <= 4'b1010; // NOR
                6'b101010: ALUControl <= 4'b1000; // SLT
                6'b101011: ALUControl <= 4'b1001; // SLTU
                6'b000000: ALUControl <= 4'b0101; // SLL
                6'b000010: ALUControl <= 4'b0110; // SRL
                6'b000011: ALUControl <= 4'b0111; // SRA
                6'b000100: ALUControl <= 4'b1011; // SLLV
                6'b000110: ALUControl <= 4'b1100; // SRLV
                6'b000111: ALUControl <= 4'b1101; // SRAV
            6'b001000: begin // JR
                temp <= 8'b00000010;
                ALUControl <= 4'b0000;
                JR <= 1'b1;
            end
        end
    endcase
end
```

```

        end
    endcase
end
6'b100011: begin // LW
    temp <= 8'b10100100;
    ALUControl <= 4'b0000;
end
6'b101011: begin // SW
    temp <= 8'b00101000;
    ALUControl <= 4'b0000;
end
6'b000100: begin // BEQ
    temp <= 8'b00010000;
    ALUControl <= 4'b0001;
end
6'b000101: begin // BNE
    temp <= 8'b00010001;
    ALUControl <= 4'b0001;
end
6'b001000: begin // ADDI
    temp <= 8'b10100000;
    ALUControl <= 4'b0000;
end
6'b001001: begin // ADDIU
    temp <= 8'b10100000;
    ALUControl <= 4'b0000;
end
6'b001100: begin // ANDI
    temp <= 8'b10100000;
    ALUControl <= 4'b0010;
end
6'b001101: begin // ORI
    temp <= 8'b10100000;
    ALUControl <= 4'b0011;
end
6'b001110: begin // XORI
    temp <= 8'b10100000;
    ALUControl <= 4'b0100;
end
6'b001010: begin // SLTI
    temp <= 8'b10100000;
    ALUControl <= 4'b1000;
end
6'b001011: begin // SLTIU
    temp <= 8'b10100000;
    ALUControl <= 4'b1001;
end
6'b000010: begin // J
    temp <= 8'b00000010;
    ALUControl <= 4'b0010;

```



```
end
6'b000011: begin // JAL
    temp <= 8'b11000010;
    ALUControl <= 4'b0010;
    JAL <= 1'b1;
end
6'b001111: begin // LJL
    temp <= 8'b10100000;
    ALUControl <= 4'b1110;
end
default: temp <= 8'bxxxxxxxx; // NOP
endcase
{RegWrite,RegDst,ALUSrc,Branch,MemWrite,MemtoReg,Jump,BNE} = temp;
end

assign PCSrc = Branch & (Zero ^ BNE);

endmodule
```